

Flappy Learner: A Reinforcement Learning Project

Winter in Data Science (WiDS)

K. Ajay Teja

Contents

1	Introduction	2
2	Learning Phase: Weeks 1–6	2
3	Flappy Bird Environment Design (Week 7)	2
3.1	Environment Overview	2
3.2	State Space	2
3.3	Action Space	3
3.4	Reward Function	3
4	DQN Agent and Training Setup (Week 8)	3
4.1	Network Architecture	3
4.2	Training Configuration	3
5	Experimental Results and Analysis	4
5.1	Episode Reward Progression	4
5.2	Moving Average Reward	5
5.3	Exploration Decay	6
5.4	Q-Value Evolution	7
5.5	Training Loss	8
5.6	Weight Distribution Analysis	9
5.7	Reward Distribution	10
6	Conclusion	10

1 Introduction

Reinforcement Learning (RL) is a learning paradigm in which an agent learns to make sequential decisions by interacting with an environment and receiving feedback in the form of rewards. Unlike supervised learning, RL does not rely on labeled datasets; instead, learning emerges through trial-and-error interactions.

The objective of this project was to move beyond theoretical understanding and apply reinforcement learning techniques to a non-trivial interactive environment. Flappy Bird was chosen due to its simple rules but challenging dynamics, requiring precise control and long-term decision making.

This project was completed as part of the **Winter in Data Science (WiDS)** program. The work was structured across eight weeks, with the first six weeks focused on learning reinforcement learning fundamentals and the final two weeks dedicated to designing and training the Flappy Learner agent.

2 Learning Phase: Weeks 1–6

The initial phase of the project focused on building a strong foundation in reinforcement learning theory and algorithms. Core concepts studied during this phase include Markov Decision Processes (MDPs), reward formulation, value functions, and policy optimization.

Algorithms explored include Monte Carlo methods, Temporal Difference learning, SARSA, Q-learning, Deep Q-Networks (DQN), and policy gradient methods such as Proximal Policy Optimization (PPO). These methods were implemented and tested on smaller environments, enabling a deeper understanding of convergence behavior, stability, and exploration strategies.

The insights gained during this phase directly influenced the design choices made in the final project, particularly the selection of a value-based DQN approach for a discrete-action game environment.

3 Flappy Bird Environment Design (Week 7)

3.1 Environment Overview

A custom Flappy Bird environment was implemented using the Pygame library. The environment was structured to be compatible with reinforcement learning agents by exposing standard interfaces such as `reset()`, `step(action)`, and `render()`.

3.2 State Space

The agent observes a four-dimensional continuous state vector:

- Bird vertical position (normalized)
- Bird vertical velocity (normalized)

- Horizontal distance to the next pipe (normalized)
- Vertical difference between the bird and the pipe gap center (normalized)

This representation captures the essential dynamics of the game while remaining computationally efficient.

3.3 Action Space

The action space is discrete and consists of two actions:

- Action 0: No flap
- Action 1: Flap

3.4 Reward Function

The reward function was designed to encourage survival and successful navigation:

- +0.1 for each timestep survived
- +1.0 for passing a pipe
- -10.0 for collision or leaving the screen

This reward shaping provides frequent feedback while strongly penalizing failure.

4 DQN Agent and Training Setup (Week 8)

4.1 Network Architecture

The Deep Q-Network consists of two hidden layers with 128 neurons each and ReLU activations. The output layer produces Q-values corresponding to the two possible actions.

4.2 Training Configuration

The agent was trained with the following configuration:

- Total training episodes: 300
- Replay buffer size: 50,000
- Batch size: 64
- Learning rate: 0.001
- Discount factor γ : 0.99
- ϵ decay: $1.0 \rightarrow$ approximately 0.22

An ϵ -greedy strategy was used to balance exploration and exploitation during training.

5 Experimental Results and Analysis

5.1 Episode Reward Progression

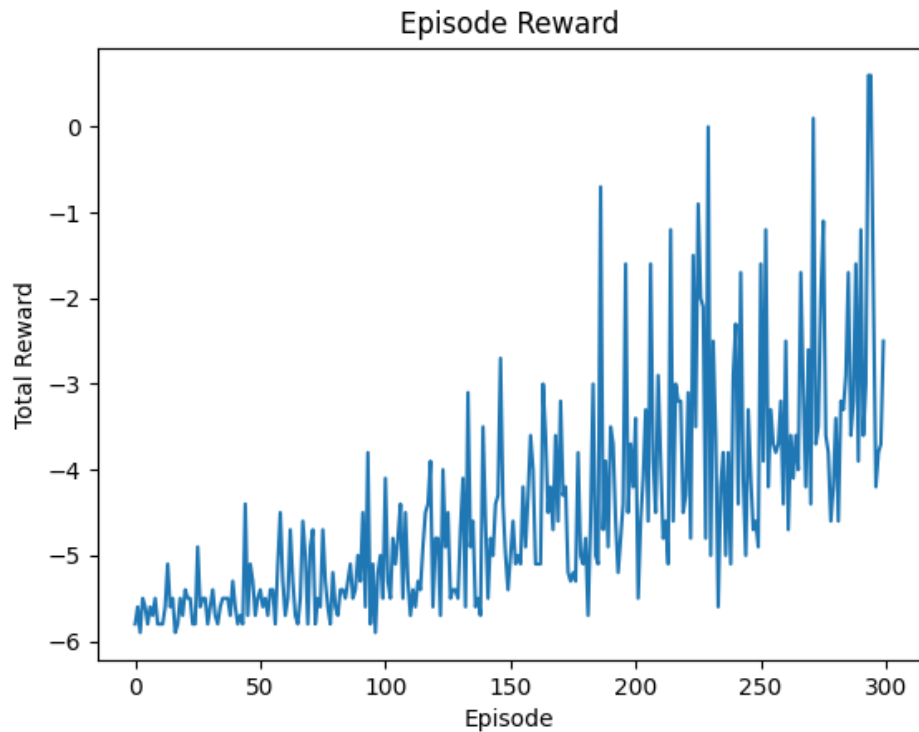


Figure 1: Episode-wise reward across 300 training episodes.

The episode rewards initially cluster around -5.8 and gradually improve over time. Several later episodes achieve near-zero or positive rewards, indicating increased survival time.

5.2 Moving Average Reward

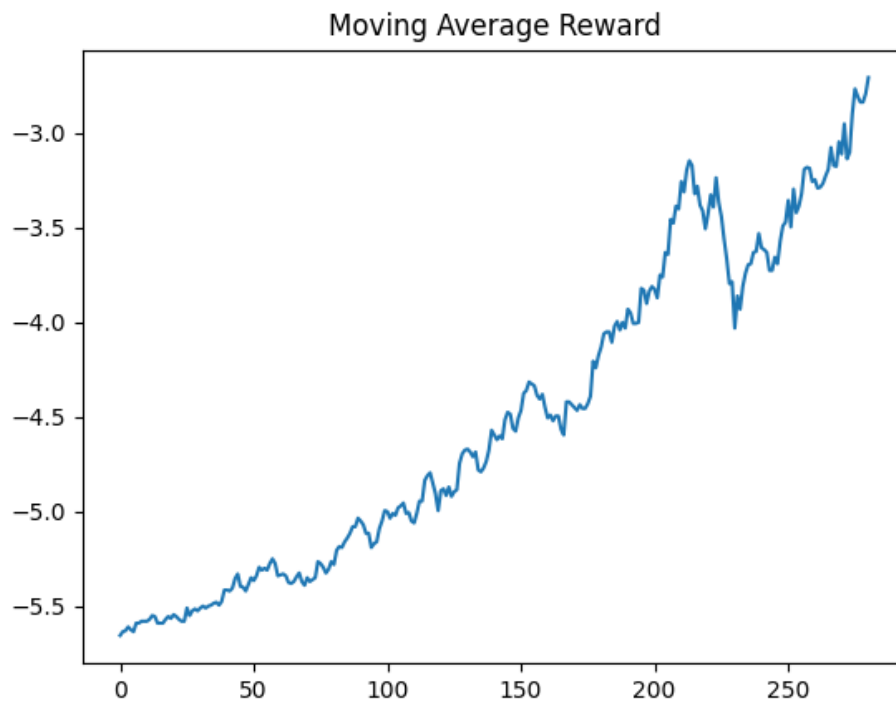


Figure 2: Moving average of episode rewards showing gradual learning progress.

The moving average increases from approximately -5.6 to around -2.8, confirming a consistent upward learning trend.

5.3 Exploration Decay

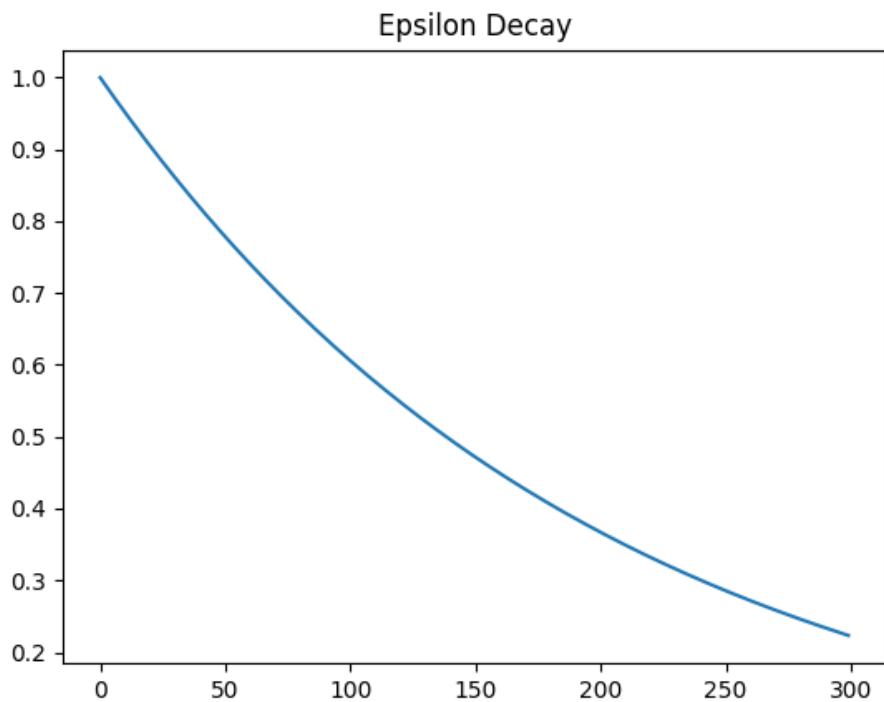


Figure 3: Decay of exploration parameter ϵ over training episodes.

The smooth decay allows the agent to transition from exploration-heavy behavior to more exploitative decision making.

5.4 Q-Value Evolution

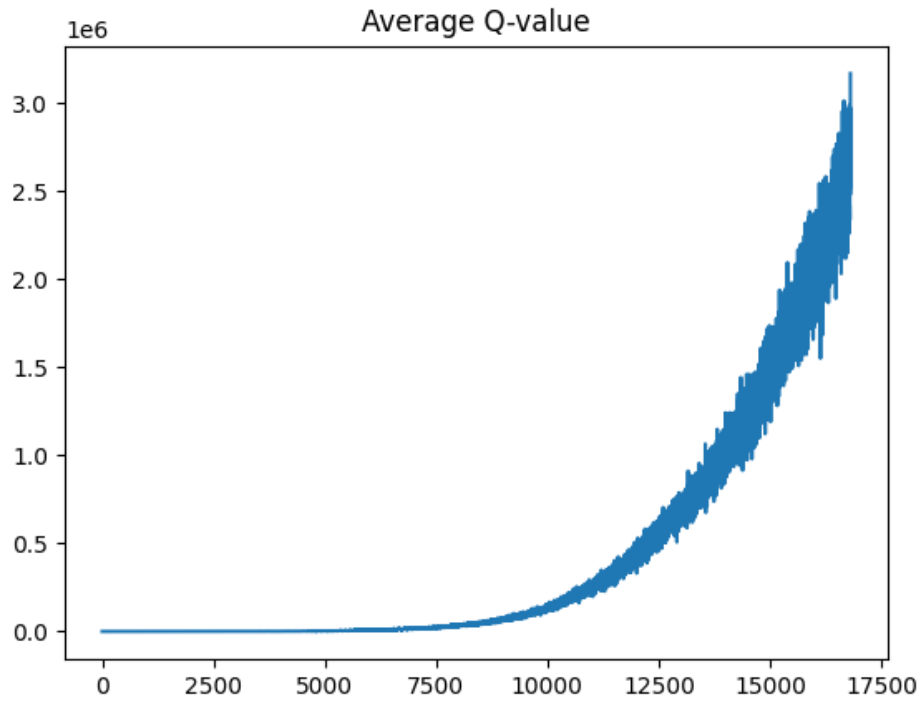


Figure 4: Average Q-value growth during training.

The rapid increase in Q-values reflects aggressive updates, highlighting instability common in vanilla DQN without a target network.

5.5 Training Loss

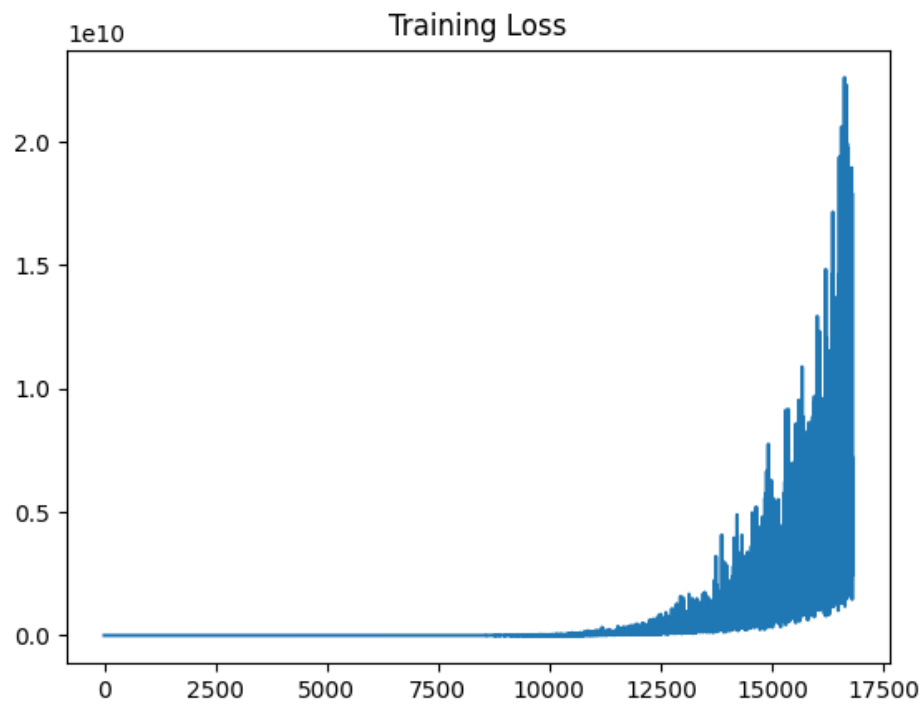


Figure 5: Training loss over time showing increasing variance.

The rising loss magnitude further indicates instability in value estimation during training.

5.6 Weight Distribution Analysis

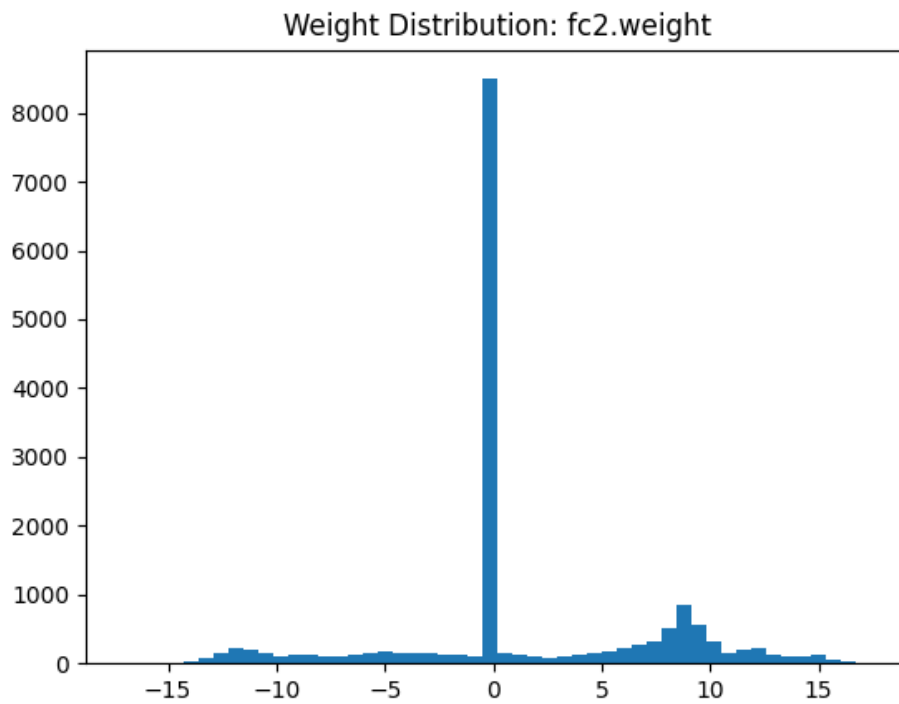


Figure 6: Distribution of learned weights in the second hidden layer.

The distribution shows a strong concentration near zero with long tails, indicating sparse but specialized feature learning.

5.7 Reward Distribution

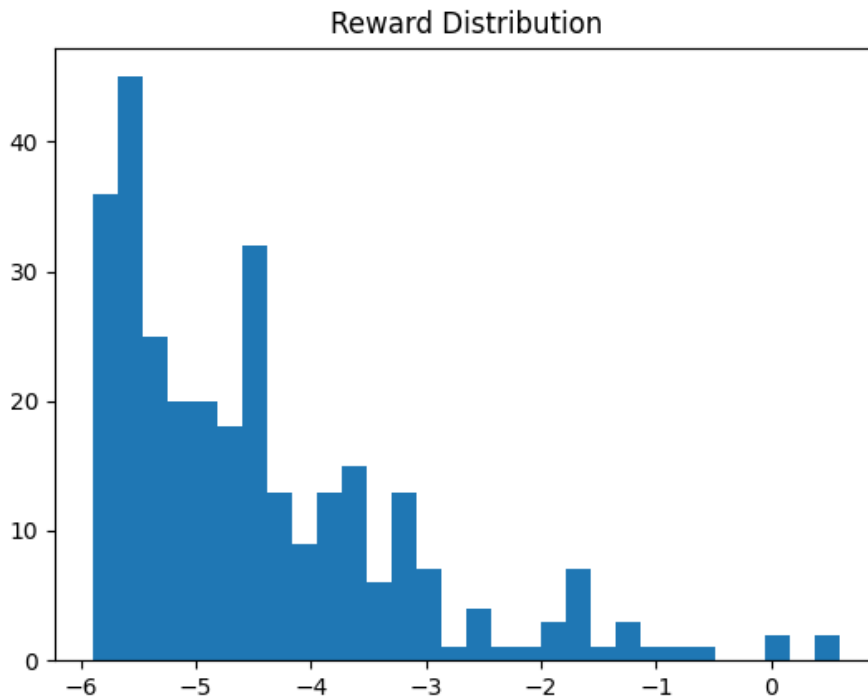


Figure 7: Histogram of episode rewards obtained during training.

The distribution shifts toward higher rewards, demonstrating reduced frequency of catastrophic failures.

6 Conclusion

The Flappy Learner project successfully demonstrates the complete reinforcement learning workflow, from foundational learning to environment design and agent training. Although training exhibited instability due to the absence of advanced stabilization techniques, the agent showed clear improvement in performance over time.

This project provided practical experience in reinforcement learning system design, debugging deep learning pipelines, and interpreting empirical results, fulfilling the objectives of the Winter in Data Science (WiDS) program.